Computer Vision Assignment <1>

Team members

Name	ID
Mohamed Hany Ibrahim	8687
Mazen Tarek Adel	8963
Yassin Ashraf	8870

Part 1: Filters and Edges

This part builds 2D convolution from scratch and uses it to find edges. Every filter in the assignment (derivatives, Gaussians, pyramids) is a convolution, so 1.1 is the foundation for everything that follows.

1.1 Convolution from Scratch

What we implemented

Convolving with a filter is the same as correlating with the filter flipped along both axes, so every version first flips the kernel with `kernel[::-1, ::-1]`. All versions pad the input with zeros (`np.pad`, constant mode) by `kh // 2` rows and `kw // 2` columns on each side, so the output has the same size as the input. Filters have odd heights and widths.

1. **Loop version (`conv2d_loop`).** Four nested loops: over every output pixel (y, x) and every filter entry (i, j). For each pixel it sums `padded[y+i, x+j] * flipped_kernel[i, j]`. This is the direct definition of convolution.
2. **Vectorized version (`conv2d_vectorized`).** There is no loop over the pixels. We loop only over the filter entries, and for each entry we add a shifted copy of the whole padded image, scaled by that entry: `out += kf[i, j] * padded[i:i+H, j:j+W]`. Each addition is one whole-image NumPy operation.
3. **Separable version (`conv2d_separable`).** `separable_factors` tests whether the filter has rank 1 using its SVD: the kernel is separable if the second singular value is zero up to rounding error (`S[1] <= 1e-10 * S[0]`). If it is, the kernel equals the outer product of a column and a row, built from the first singular vectors scaled by the square root of the first singular value. We convolve with the column (vertical pass), then with the row (horizontal pass). If the rank is larger than 1 we fall back to the vectorized version.

```python
def conv2d_loop(img, kernel, pad_mode="constant"):
    """Version 1: loops over every output pixel AND every filter entry."""
    img = np.asarray(img, dtype=np.float64)
    kernel = np.asarray(kernel, dtype=np.float64)
    H, W = img.shape
    kh, kw = kernel.shape
    padded = _pad(img, kh // 2, kw // 2, pad_mode)
    kf = kernel[::-1, ::-1]  # convolution = correlation with the flipped filter
    out = np.zeros((H, W))
    for y in range(H):
        for x in range(W):
            s = 0.0
            for i in range(kh):
                for j in range(kw):
                    s += padded[y + i, x + j] * kf[i, j]
            out[y, x] = s
    return out
```

```python
def conv2d_vectorized(img, kernel, pad_mode="constant"):
    """Version 2: no loop over pixels. Loop over the filter entries only and add a
    shifted copy of the whole image, scaled by that filter entry."""
    img = np.asarray(img, dtype=np.float64)
    kernel = np.asarray(kernel, dtype=np.float64)
    H, W = img.shape
    kh, kw = kernel.shape
    padded = _pad(img, kh // 2, kw // 2, pad_mode)
    kf = kernel[::-1, ::-1] #explaination: the ::1 means to take every element in the array
    out = np.zeros((H, W))
    for i in range(kh):
        for j in range(kw):
            out += kf[i, j] * padded[i:i + H, j:j + W]
    return out
```

```python
def separable_factors(kernel, tol=1e-10):
    """SVD rank-1 test. Returns (column, row) if the kernel is separable up to
    rounding error, otherwise None."""
    kernel = np.asarray(kernel, dtype=np.float64)
    U, S, Vt = np.linalg.svd(kernel)
    if S[0] == 0:  # all-zero kernel
        return np.zeros((kernel.shape[0], 1)), np.zeros((1, kernel.shape[1]))
    if len(S) > 1 and S[1] > tol * S[0]:  # second singular value not ~0 -> rank > 1
        return None
    r = np.sqrt(S[0])
    return (U[:, 0] * r).reshape(-1, 1), (Vt[0] * r).reshape(1, -1)  # kh x 1, 1 x kw


def conv2d_separable(img, kernel, pad_mode="constant"):
    """Version 3: rank-1 kernel -> convolve with the column, then with the row.
    Otherwise fall back to the vectorized version."""
    factors = separable_factors(kernel)
    if factors is None:
        return conv2d_vectorized(img, kernel, pad_mode)
    col, row = factors
    tmp = conv2d_vectorized(img, col, pad_mode)   # vertical pass
    return conv2d_vectorized(tmp, row, pad_mode)  # horizontal pass
```

Sorry, something went w

Sanity check against SciPy

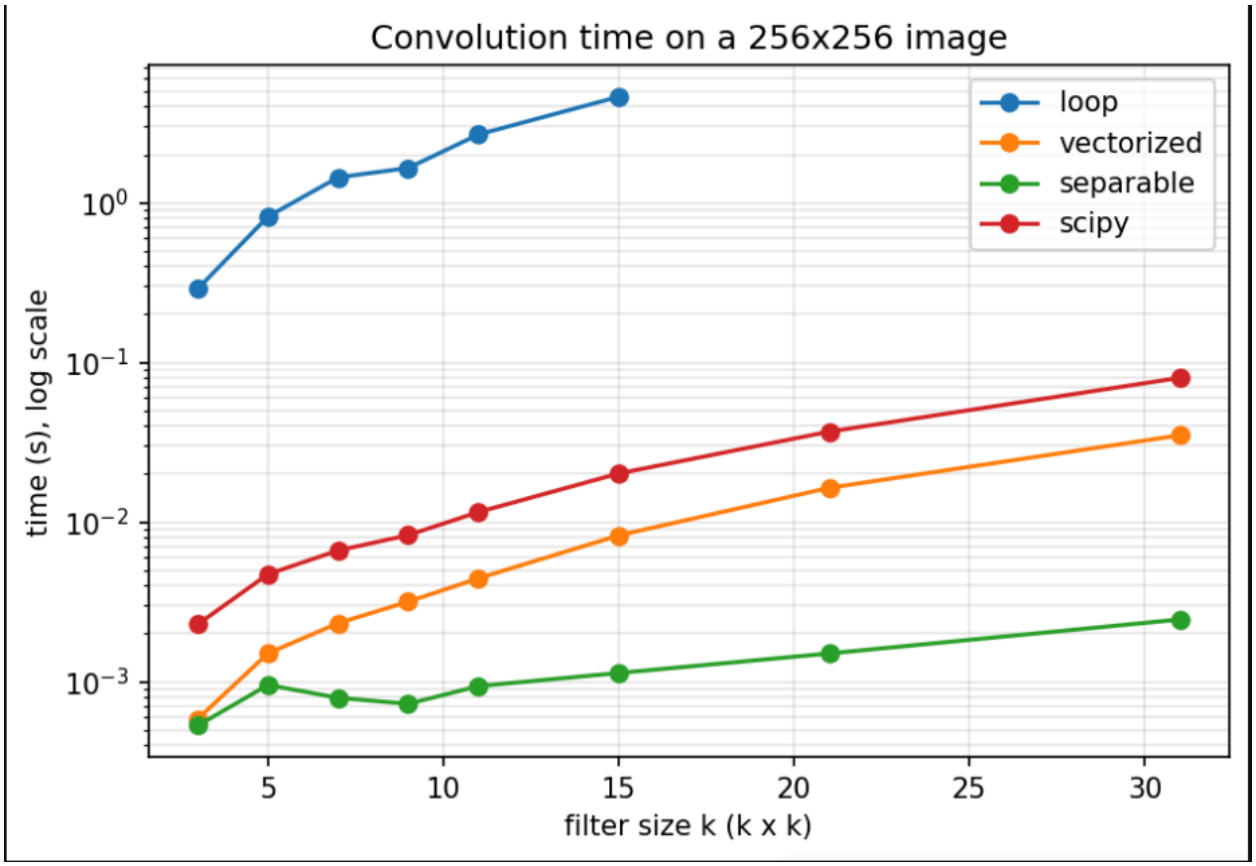
We ran all three versions on 60 small random images (heights and widths between 5 and 20) with random filters of odd height and width (1, 3, 5 or 7). Half of the filters were separable (an outer product of two random vectors) and half were general random filters. Each result was compared with `scipy.signal.convolve2d(img, k, mode="same")`, which also pads with zeros. All three versions matched in every case, with a worst absolute error of **[FILL: the worst error your run printed, about 1e-15]**, which is rounding error. The check also asserts that every outer-product kernel is detected as separable.

```
[1.1] sanity check passed: 60 random cases (30 separable, 30 general), 3 versions each, worst abs erro
r vs scipy = 8.88e-15
```

Timing

We timed the three versions, and SciPy as a reference, on a 256 x 256 image with Gaussian filters (sigma = size / 6) of sizes 3x3, 5x5, 7x7, 9x9, 11x11, 15x15, 21x21 and 31x31. Each time is the best of several runs (one run for the slow loop version).

```
[1.1] Timing on a 256x256 image, Gaussian filters (sigma = size/6), seconds
   size |       loop | vectorized |  separable |      scipy
------------------------------------------------------------
  3x3   |     0.2892 |     0.0006 |     0.0005 |     0.0023
  5x5   |     0.8186 |     0.0015 |     0.0010 |     0.0047
  7x7   |     1.4427 |     0.0023 |     0.0008 |     0.0066
  9x9   |     1.6511 |     0.0032 |     0.0007 |     0.0083
 11x11  |     2.6754 |     0.0044 |     0.0009 |     0.0115
 15x15  |     4.6143 |     0.0082 |     0.0011 |     0.0201
 21x21  |    skipped |     0.0164 |     0.0015 |     0.0368
 31x31  |    skipped |     0.0350 |     0.0025 |     0.0801
   saved part1_output\1_1_timing.png
```



Explanation of the differences

- **Loop version: slowest by far.** It does the same arithmetic as the others, H x W x kh x kw multiply-adds (about 65,000 x k^2 for our image), but every one is executed by the Python interpreter, one at a time. Its time grows with k^2 and is about two orders of magnitude above the vectorized version.
- **Vectorized version: much faster, still quadratic.** Python now loops only over the k^2 filter entries, and each iteration is a single whole-image NumPy operation that runs in compiled C. The

arithmetic is the same as in the loop version, but the interpreter overhead is gone. The time still grows roughly with k^2, because a k x k filter has k^2 entries.

- **Separable version: fastest for large filters, and it grows linearly.** A Gaussian is rank 1, so one k x k convolution is replaced by two 1D passes with k entries each. That is 2k whole-image operations instead of k^2, so the time grows with k instead of k^2. For the 31x31 filter this is about 31 / 2 times fewer operations than the vectorized version. For very small filters (3x3) the advantage disappears, because the SVD test and the two passes add a fixed overhead, so separable can be equal to or slightly slower than vectorized there.
- **SciPy.** `convolve2d` computes the direct 2D sum in C and does not exploit separability, so its time also grows with k^2. It is in the same range as our vectorized version, and slower than our separable version for large filters.

1.2 Finite Difference Operator

What we did

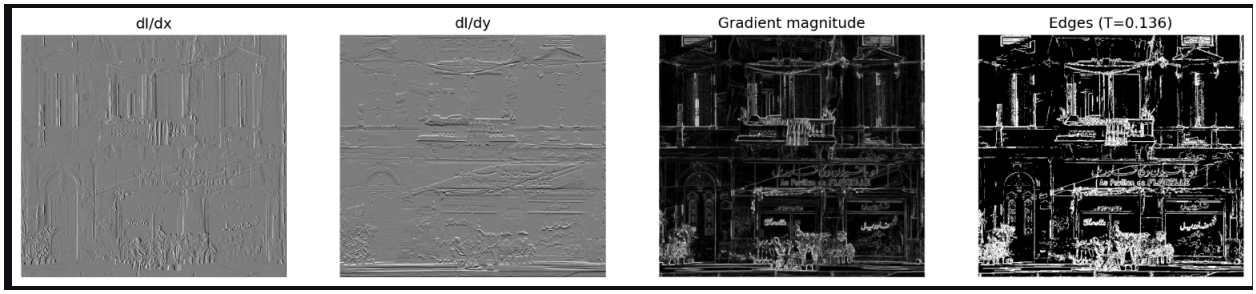
We used the grayscale image **[facade.png]** (at most 1000 pixels on the long side). With our 1.1 convolution (the separable version, as the assignment requires) we filtered it with D_x = [1 0 -1] and D_y = D_x transposed. Both are rank 1, so they run through the separable path, with zero padding as required in Part 1.

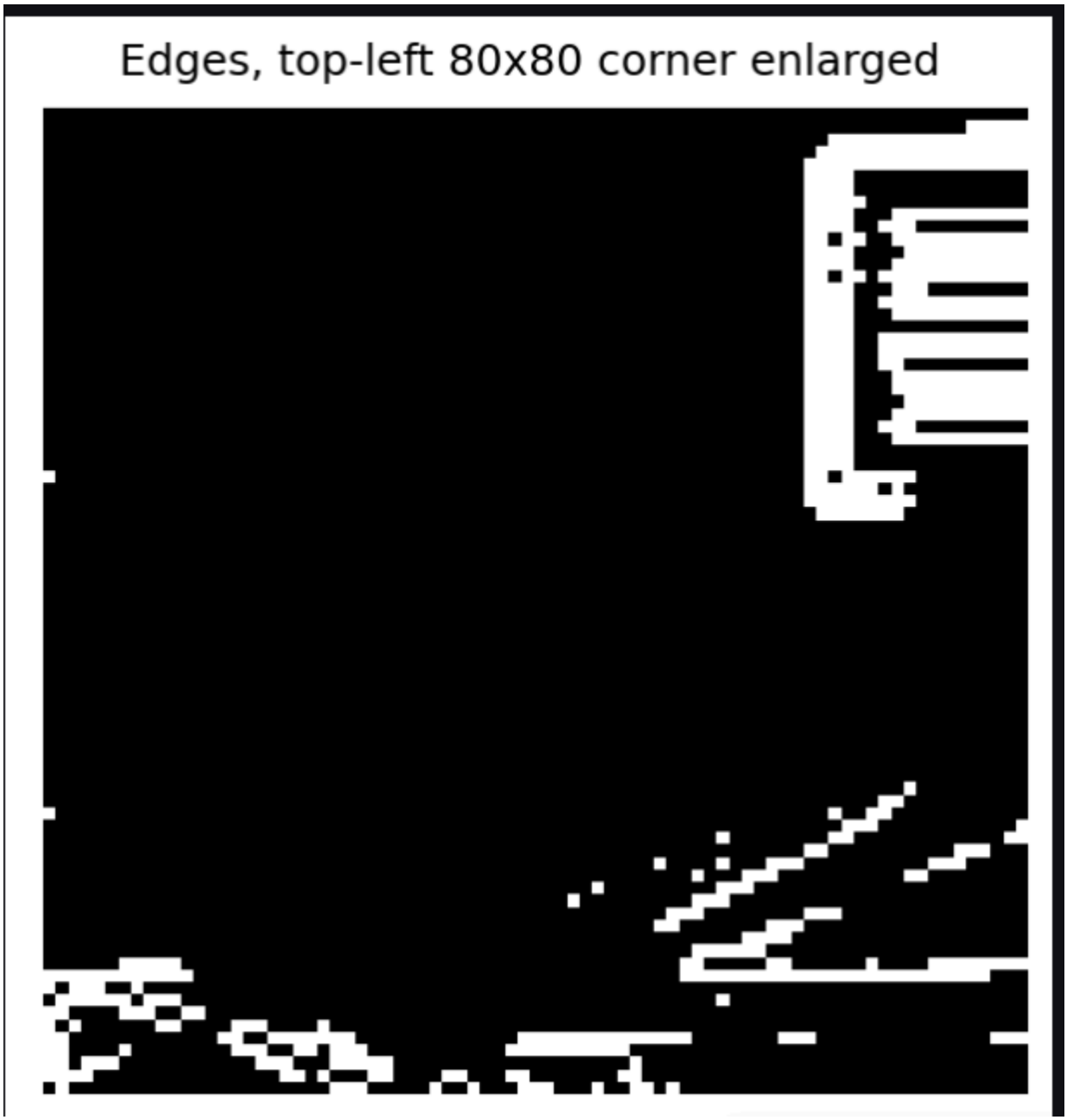
1. Derivatives: `gx = conv2d_separable(img, Dx)` and `gy = conv2d_separable(img, Dy)`.
2. Gradient magnitude: `sqrt(gx**2 + gy**2)`.
3. Edge image: the magnitude compared with a threshold T, giving 1 where the magnitude is above T and 0 elsewhere.

The two derivative images go negative, so we show them with zero as mid-gray (`0.5 + 0.5 * x / m`, clipped to [0, 1]).

```
Dx = np.array([[1.0, 0.0, -1.0]])   # 1 x 3
Dy = Dx.T                           # 3 x 1


def gradient_maps(img, kx, ky):
    gx = conv2d_separable(img, kx)
    gy = conv2d_separable(img, ky)
    return gx, gy, np.sqrt(gx ** 2 + gy ** 2)
```


dI/dx
dI/dy
Gradient magnitude
Edges (T=0.136)

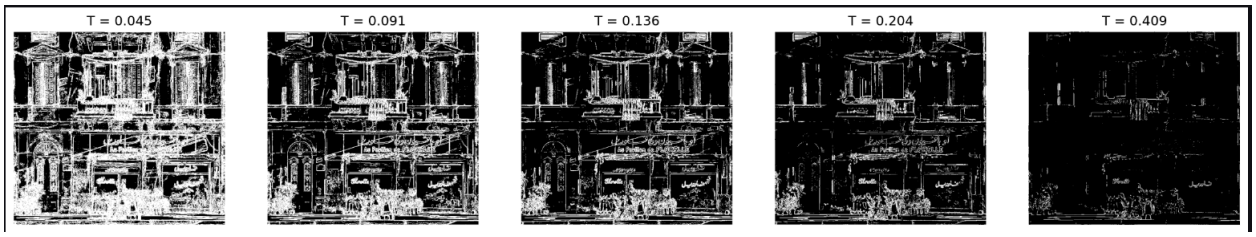

Edges, top-left 80x80 corner enlarged

Threshold

The threshold is T = **0.1363**. We set it as a fraction (**0.25**) of the strength of the strong edges, measured as the 99th percentile of the gradient magnitude. The percentile is computed on the interior only, with a 3-pixel margin excluded, because the frame lines (explained below) are artificially strong and would push the threshold up.

We justify the value by sweeping it (T/3, T/1.5, T, 1.5T, 3T):

- With a threshold that is too low, almost every textured or noisy pixel becomes an edge, and the real structure is buried in a thick mess.
- With a threshold that is too high, weak but real edges break apart or disappear.
- The chosen T keeps the main building and object contours as connected lines while dropping most of the fine texture and noise.



Where the lines along the image frame come from

The convolution pads the image with zeros. At the border, the filter therefore sees the real pixel on one side and a padded zero on the other, so the image appears to jump from its intensity down to 0. That is an artificial step edge along the whole frame, and its size is roughly the pixel intensity, which is much larger than most real gradients in the image. The magnitude is high there and the threshold keeps it, which draws a line along the frame. It is visible in the enlarged corner. Reflect padding, which we use from Part 2 on, avoids this because the padded values are copies of the image next to the border.

1.3 Derivative of Gaussian (DoG) Filter

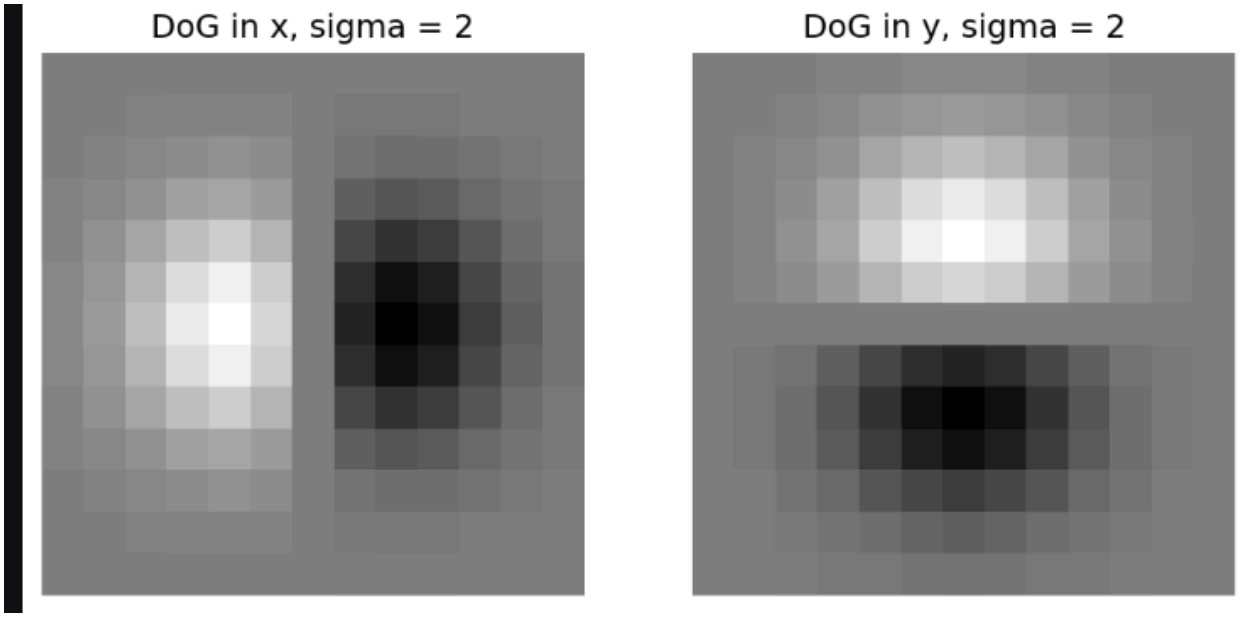
What we did

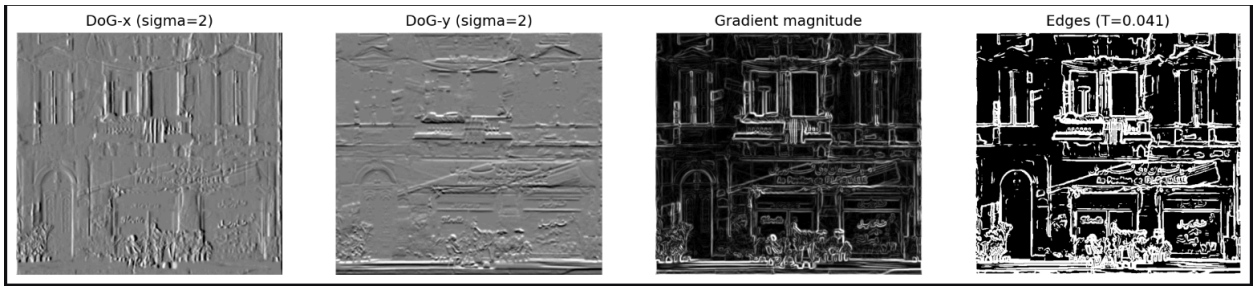
Derivatives amplify noise, so we blur before differentiating. Convolution is associative, so (I * G) * D = I * (G * D): the blur and the derivative can be folded into one filter.

1. Build a 2D Gaussian kernel G with sigma =2.0, of size 2 * ceil(3 * sigma) + 1(**13 x 13 for sigma=2**), which covers plus and minus 3 sigma, normalized to sum to 1.
2. Filter the kernel itself with D_x and with D_y using our convolution. The two results are DoG_x and DoG_y.

3. Apply each DoG filter to the image with a single convolution, take the gradient magnitude, and threshold it as in 1.2. Both DoG filters are rank 1 (the program checks it), so they also run through the separable path.
4. As a check, the program compares (I * G) * D_x with I * DoG_x away from the border. They agree up to **1.232**, which is only the truncation of the kernel.

```
def dog_filters(sigma):
    size = 2 * int(np.ceil(3 * sigma)) + 1
    G = gaussian_kernel(size, sigma)
    DoGx = conv2d_separable(G, Dx)   # filter the kernel itself with Dx
    DoGy = conv2d_separable(G, Dy)   # ... and with Dy
    return G, DoGx, DoGy
```





Parameters

sigma = **2.0** and threshold T =**0.1234**, chosen the same way as in 1.2 (a fraction **0.25** of the 99th percentile of the interior magnitude). The threshold is lower than in 1.2 because blurring spreads each edge out and lowers the peak gradient, so the magnitudes are smaller overall.

How and why the maps differ from 1.2

- **Less noise.** A derivative filter has a gain that grows with frequency, so it amplifies noise and fine texture. The Gaussian is a low-pass filter that removes those high frequencies before they are differentiated. In 1.2 the edge image is full of small speckles and broken fragments in textured regions; the DoG edges are far cleaner.
- **Smoother, wider responses.** The DoG filter is about 13 pixels wide instead of 3, so each edge is spread over several pixels. The derivative maps look smoother and the edges are thicker and more continuous.
- **Only the strong, larger-scale edges survive.** Fine details smaller than sigma are blurred away, so the DoG result keeps the main contours of the scene.
- **Less precise localization.** The price of the blur is that edges are less sharply localized and nearby edges can merge.
- **The frame lines are wider.** The image is still zero padded, and the DoG filter is wider, so the artificial border response spreads over a band of about the filter radius instead of one or two pixels.

The DoG filters themselves look like what the theory predicts: DoG_x is a Gaussian blob split into a positive and a negative lobe along x (a smoothed difference between left and right), and DoG_y is the same along y.

Part 2: Frequencies, Pyramids and Blending

Downsampling & Aliasing

Goal

we want to see what happens to fine detail when an image is **shrunk down** (downsampled). We will see that:

1. Naively keeping every 4th pixel produces **false circles** — high frequencies **alias** into low ones.
2. Gaussian-blurring **before** decimating prevents this. The blur acts as an **anti-aliasing low-pass filter**.
3. tune σ to find the **smallest** value at which the false circles disappear.

Finally, we connect this to **Gaussian pyramid** construction: every level must blur *before* it subsamples.

Theoritical

Gaussian kernel

A 1-D Gaussian with standard deviation σ:

$$G_\sigma(x) = \frac{1}{\sqrt{2\pi}\,\sigma} exp\left(-\frac{x^2}{2\sigma^2}\right)$$

2-D Gaussian blur (separable)

Because the Gaussian is separable, blurring an image I reduces to two 1-D convolutions:

$$\left(I * G_\sigma\right)(u,v) = \sum_x \sum_y I(u-x,\, v-y)\, G_\sigma(x)\, G_\sigma(y)$$

Downsampling by factor N

Let I be the original image, of size $H \times W$ (height × width, in pixels). The pixel at row i and column j is written $I(i,j)$. We want a smaller image that represents the same scene but sampled on a **coarser grid(less pixels)** — we'll naievely take the N-th pixel in each direction.

The naive formula

$$I_{\downarrow N}(i,j) = I(Ni,\ Nj), \qquad i = 0,\ldots,\lfloor H/N \rfloor - 1,\quad j = 0,\ldots,\lfloor W/N \rfloor - 1$$

Read it piece by piece:

- $I_{\downarrow N}$ is the *output* image. The subscript $\downarrow N$ is standard shorthand for "downsampled by a factor of N". For our experiment $N = 4$.
- (i, j) are coordinates **in the smaller output image**, not the original. They run from 0 up to $\lfloor H/N \rfloor - 1$ rows and $\lfloor W/N \rfloor - 1$ columns. For a 512×512 input with $N = 4$, the output is 128×128.
- $I(Ni, Nj)$ says: to fill output pixel (i, j), look at the **original** pixel at row Ni and column Nj — i.e. jump by N each time. Only those pixels are copied; all others are **thrown away**.

In NumPy this is literally `image[::N, ::N]`. The step size of `N` in the slice is what does the "keep every N-th pixel" operation.

Why it produces aliasing. The original image is sampled at a rate f_s. The output image has a sampling rate of f_s/N — i.e. it sees the scene through a coarser grid. By the Nyquist–Shannon sampling theorem, the coarser grid can only faithfully represent frequencies below

$$f_{Nyq}^{new} = \frac{f_s}{2N}.$$

Any content in I above that limit is **not removed** by just reming. Instead, when we resample on the coarse grid, those high frequencies get **folded** (aliased) down into lower frequencies and appear as *false* structure — the false circles / moiré we see with `rings.png`. That is why naive removal is unsafe.

The anti-aliased formula

$$I_{\downarrow N}^{AA} = (I * G_\sigma)_{\downarrow N}$$

Read it inside-out:

- G_σ is a Gaussian kernel with standard deviation σ — the same 1-D Gaussian from the theory section, extended to 2-D separably.
- $I * G_\sigma$ is the image **blurred** by that Gaussian. Convolution with a Gaussian is a *low-pass filter*: it keeps low frequencies and suppresses high ones. The larger σ, the lower the cutoff frequency — i.e. the more aggressive the smoothing.
- $(\cdot)_{\downarrow N}$ is then exactly the naive decimation formula above, applied to the blurred image.

So the anti-aliased method is: **blur first, then decimate.** The blur's job is to *pre-remove* all content above f_{Nyq}^{new} — the frequencies that would otherwise alias. After the blur, the surviving frequencies are already below the new Nyquist limit, so the subsequent decimation introduces no folding. The result is a smaller image that is smoother but **faithful**: no false structure is created.

Choosing σ

To suppress everything above $f_{Nyq}^{new} = f_s/(2N)$, the Gaussian's cutoff must sit at or below that frequency. The 1-D Gaussian's frequency response is itself Gaussian, with standard deviation $1/(2\pi\sigma)$ in *normalized* frequency. Requiring its effective cutoff to sit below $1/(2N)$ (in cycles per original pixel) gives the rule of thumb

$$\sigma \gtrsim \frac{N}{2}.$$

For $N = 4$ this is $\sigma \gtrsim 2$ — which is exactly why the sweep in this notebook concentrates on $\sigma \in \{2,\ 2.5,\ 3,\ 3.5,\ 4\}$. Too small a σ leaves aliasing residue; too large a σ washes out legitimate detail.

Worked example ($N = 4$, first few output pixels)

Original rows/cols kept:

Output pixel (i, j)	Original pixel $I(Ni, Nj)$
$(0, 0)$	$I(0, 0)$
$(0, 1)$	$I(0, 4)$
$(0, 2)$	$I(0, 8)$
$(1, 0)$	$I(4, 0)$
$(1, 1)$	$I(4, 4)$

Every pixel whose row or column index is **not** a multiple of 4 is discarded. For a 512×512 image that means 15 out of every 16 pixels never influence the output — hence why a single high-frequency feature can be completely misread by the coarse grid, and why pre-blurring (which lets *neighboring* pixels influence the kept ones) is essential.

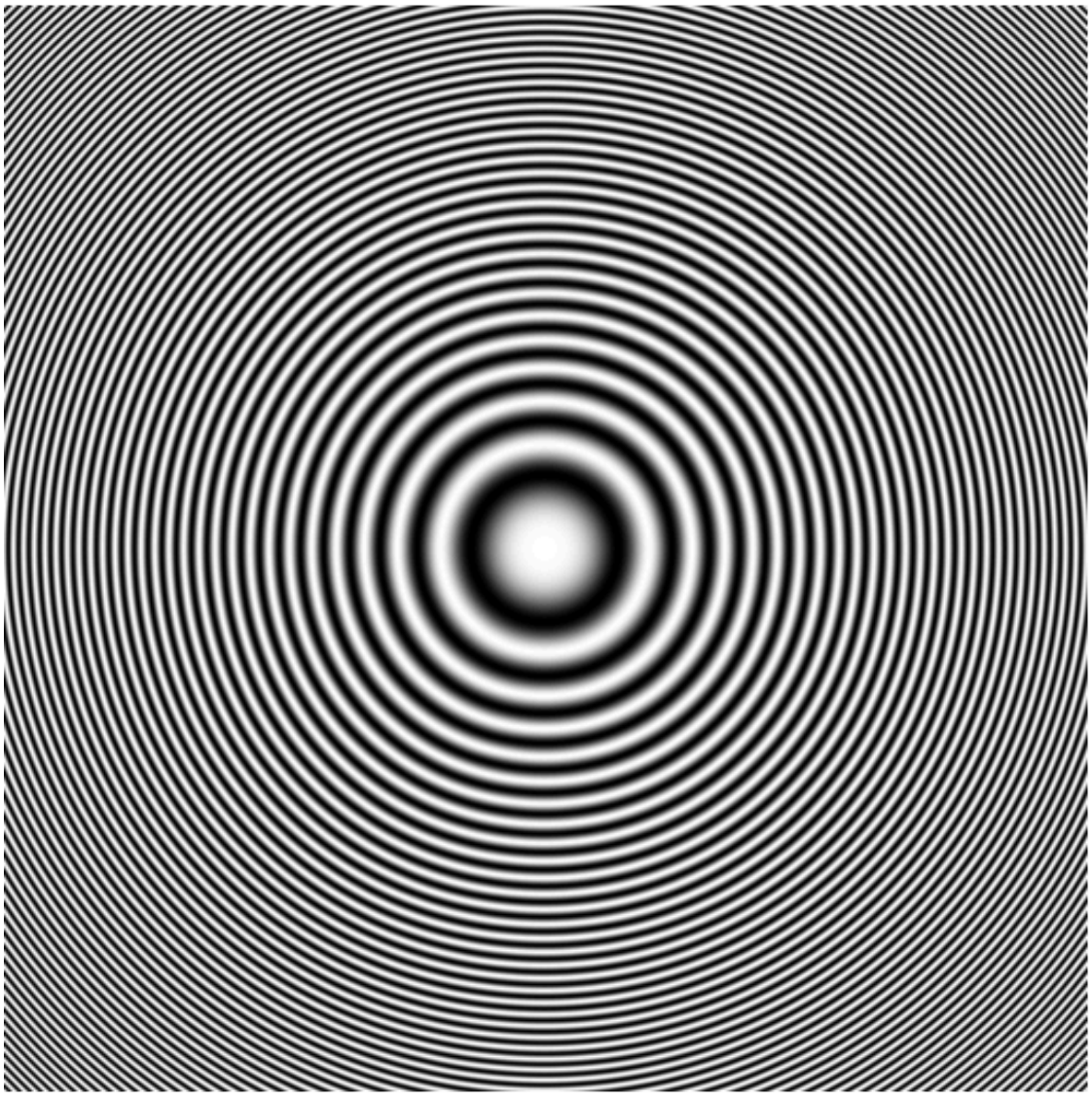
Summary

- **Naive:** $I_{\downarrow N}(i, j) = I(Ni, Nj)$ — sample the original on the coarse grid, discard the rest.
- **Anti-aliased:** $I_{\downarrow N}^{AA} = \left(I * G_\sigma\right)_{\downarrow N}$ — smooth out everything the coarse grid can't represent, *then* sample.
- The blur is not cosmetic; it is a mathematically required pre-filter to satisfy the Nyquist condition of the coarser grid.
- Rule of thumb for the Gaussian's width: $\sigma \approx N/2$ or a bit more.

---

2 — Load `rings.png` and view at 100% zoom

We display the original at its native pixel scale so nothing is hidden by an on-screen resize. Any viewer that shrinks the image for you is doing the same downsampling we are about to study — and therefore also producing false circles.



alt text

---

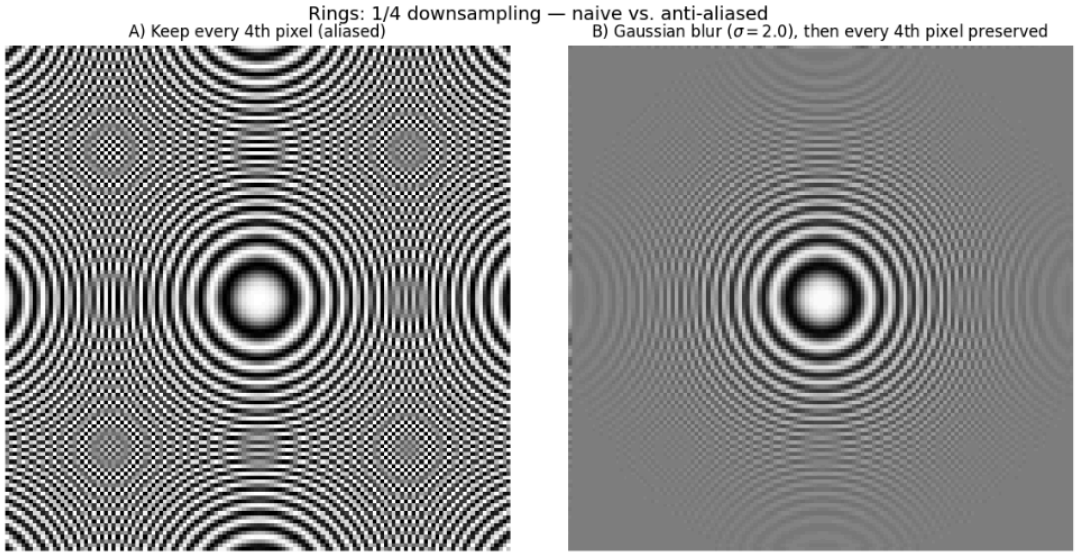
3 — Two ways to shrink by 4×

We produce two 1/4-size versions:

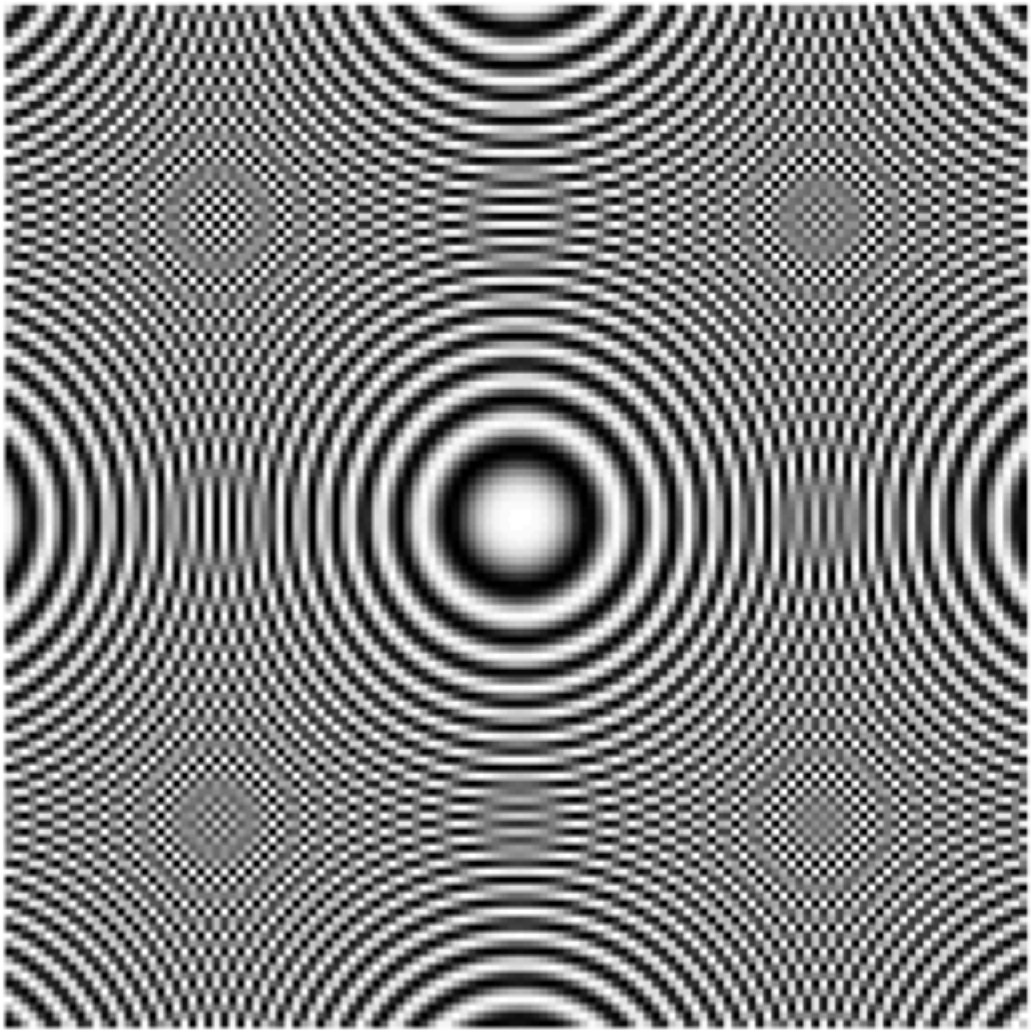
1. **Naive** — keep every 4th pixel in each direction: $I_{\downarrow 4}(i, j) = I(4i, 4j)$

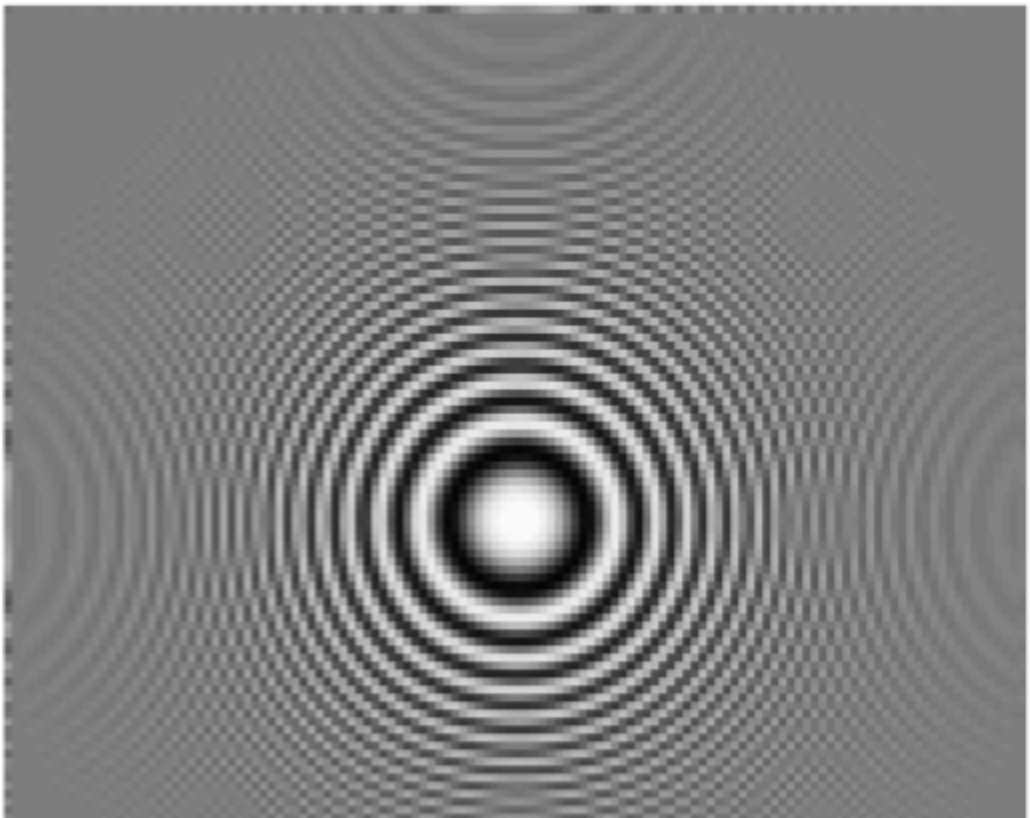
2. **Anti-aliased** — Gaussian blur first, then keep every 4th pixel: $I^{AA}_{\downarrow 4} = \left(I * G_{\sigma}\right)_{\downarrow 4}$

Both reduce $512 \times 512 \rightarrow 128 \times 128$.



alt text





alt text

What we see.

- **Naive (A)** — false circles / moiré. The fine ring pattern contains frequencies above the new Nyquist limit $f_s/8$; keeping every 4th pixel **folds** those frequencies down into low ones. The moiré we see was never in the original image.
- **Blurred (B)** — clean. The Gaussian removed content above $f_s/8$ *before* we sampled on the coarser grid, so nothing folds down. The result is smoother but faithful.

4 — σ tuning: find the smallest σ that removes false circles

We test $\sigma \in \{2,\ 2.5,\ 3,\ 3.5,\ 4\}$ (plus $\sigma = 0$, i.e. the naive case, as a baseline). The goal is to find the **smallest** σ at which the false circles caused by aliasing are no longer visible.

The HF-energy metric

For each σ we compute a scalar score:

Read it piece by piece:

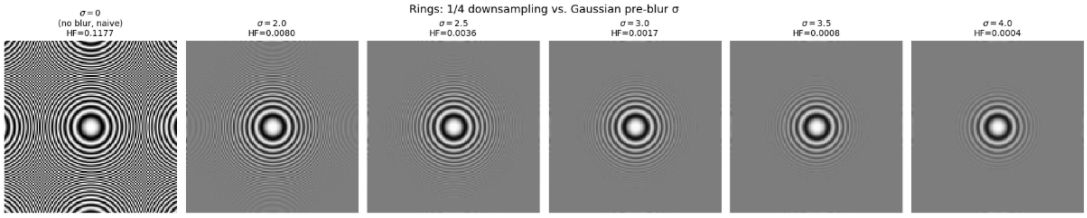
- $I_{\downarrow 4}$ is the downsampled image we are scoring.
- $\bar{I}_{\downarrow 4,\,3\times 3}(i,j)$ is the **average of the 3×3 neighbourhood** centred on pixel (i,j) (with reflect padding at the borders).
- $I_{\downarrow 4}(i,j) - \bar{I}_{3\times 3}(i,j)$ is therefore "**how much this pixel differs from its local average**" — a signed residual.
- Squaring it makes large deviations count much more than small ones (and removes the sign).
- Averaging over all $H \times W$ pixels gives a single number: the **mean squared local deviation**.

What it measures (and what it does *not*)

This is a **local-roughness / local-variance** score. Intuition:

- If a region of the image is **smooth**, each pixel is close to the average of its neighbours → small residual → small contribution.
- If a region has **rapid pixel-to-pixel fluctuation** (moiré, false circles, fine high-frequency texture), each pixel differs strongly from its neighbours → large residual → large contribution.

So a **high** HF-energy value means the image still contains a lot of pixel-scale fluctuation — exactly what aliasing looks like.



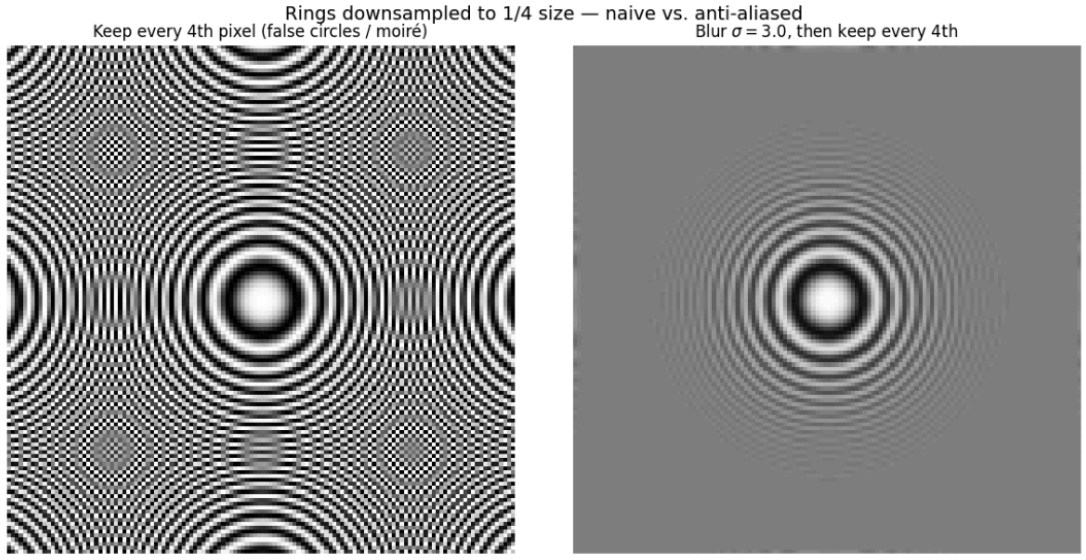

```
sigma sweep results (higher HF energy = more residual false pattern):
  sigma =  0.0   HF energy = 0.1177
  sigma =  2.0   HF energy = 0.0080
  sigma =  2.5   HF energy = 0.0036
  sigma =  3.0   HF energy = 0.0017
  sigma =  3.5   HF energy = 0.0008
  sigma =  4.0   HF energy = 0.0004
```

alt text

Interpretation of the sweep.

- $\sigma = 0$ (naive): high HF score and obvious false circles.
- $\sigma = 2$: many of the false circles are already gone, but faint ones can still be visible.
- $\sigma = 2.5$–3: the false circles are no longer visible; the HF score has dropped sharply and the ring pattern is still recognizable.
- $\sigma = 4$: very clean, but the *real* ring detail begins to wash out — this is over-blurring.

For a **factor-4** reduction, the rule of thumb is $\sigma \approx N/2 = 2$ for the *minimum* anti-aliasing and up to $\sigma \approx 3$ for a comfortable margin.



alt text

2.3 — Gaussian and Laplacian Pyramids

Build Gaussian and Laplacian pyramids **from scratch**

- Blur kernel: $\frac{1}{16}[1,\ 4,\ 6,\ 4,\ 1]$ applied separably along x and y.
- `reduce`: blur, then keep every other row/column.
- `expand`: place each pixel at the top-left of a 2×2 zero block, blur, multiply by 4.
- Reconstruct by walking the Laplacian pyramid from the coarsest level up.
- Discuss exactness and the Gaussian pyramid's total size.

0 — Theory

kernel

$$k = \frac{1}{16}[1,\ 4,\ 6,\ 4,\ 1]$$

reduce

$$G_{n+1} = \left(G_n * k\,k^{\top}\right)_{\downarrow 2}$$

expand

$$expand(J) = 4 \cdot \left(upsample_2(J) * k\,k^{\top}\right)$$

Gaussian pyramid

$$G_0 = I, \qquad G_{n+1} = reduce\left(G_n\right)$$

Laplacian pyramid

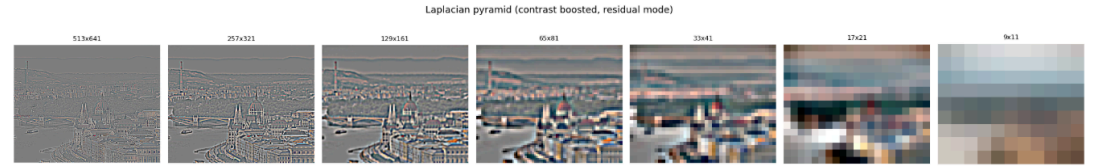
$$L_n = G_n - expand\left(G_{n+1}\right), \qquad L_L = G_L$$

Reconstruction

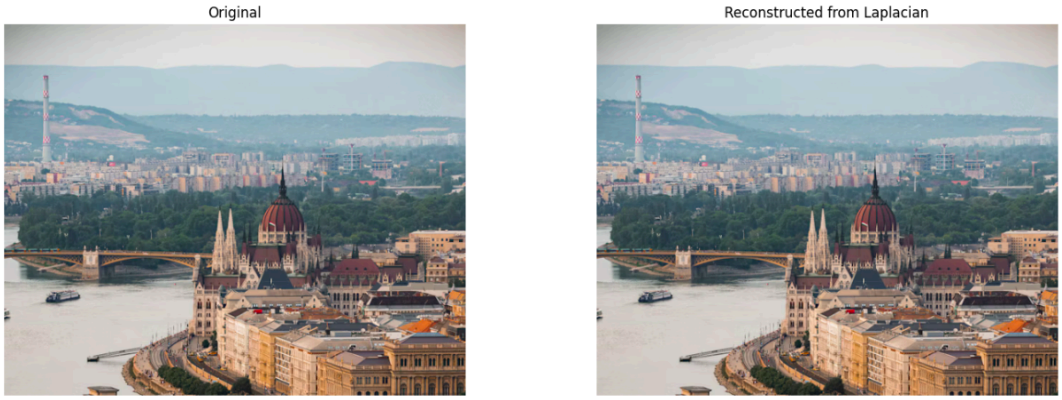
$$\hat{G}_L = L_L, \qquad \hat{G}_n = L_n + expand\left(\hat{G}_{n+1}\right)$$

Important: the Laplacian levels $L_0, \ldots, L_{L-1}$ are **signed residuals** centered on 0, whereas L_L is an **actual image** (the coarsest Gaussian). They must be displayed differently.

alt text



alt text



alt text

10 — Total size of the Gaussian pyramid

For an $H \times W$ image, level k has approximately $HW/4^k$ pixels, so

$$S = HW \sum_{k=0}^{L} \frac{1}{4^k} \rightarrow HW \cdot \frac{1}{1-1/4} = \frac{4}{3} HW.$$

The whole pyramid costs only ~33% more than the original image. Our measured ratio is slightly less than 4/3 because we stop at `min_side=16` (the tail of the series is truncated).

Hybrid Images: Man + Cat

Goal

Create a **hybrid image** by combining the low-frequency content of one image (a man) with the high-frequency content of another (a cat). The result looks like the man from a distance (low frequencies dominate) and like the cat up close (high frequencies dominate).

We build everything from scratch:

- Grayscale conversion using Rec. 601 luminance weights.
- 1-D Gaussian kernel and separable 2-D convolution with reflect padding.

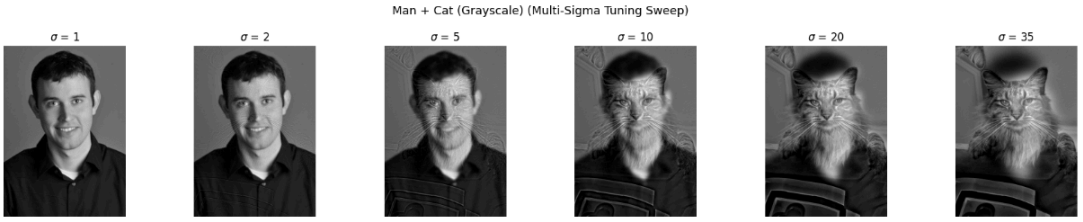
- Low-pass and high-pass filtering via Gaussian blur.
- Multi-sigma tuning sweep to pick the best cutoff.

Hybrid image equation

$$I_{hybrid} = \underbrace{I_{low} * G_{\sigma_{low}}}_{\hookrightarrow low-pass} + \underbrace{I_{high} - I_{high} * G_{\sigma_{high}}}_{\hookrightarrow high-pass}$$

Gaussian kernel

$$G(x) = \frac{exp\left(-x^2/\left(2\sigma^2\right)\right)}{\sum_{x'} exp\left(-x'^2/\left(2\sigma^2\right)\right)}$$



alt text

6 — Final result with the best sigma ($\sigma = 10$)

After running multiple iterations over [1, 2, 5, 10, 20, 35], $\sigma = 10$ was selected as the optimal cutoff for Man + Cat:

- Low σ (< 10) retains too many human edges, muddying the cat details up close.
- High σ (> 10) blurs the human silhouette too much and leaves cat ghosting artifacts.
- $\sigma = 10$ cleanly isolates close-up whiskers/eyes while preserving face shape at distance.

alt text

Multiresolution Blending with a Laplacian Pyramid

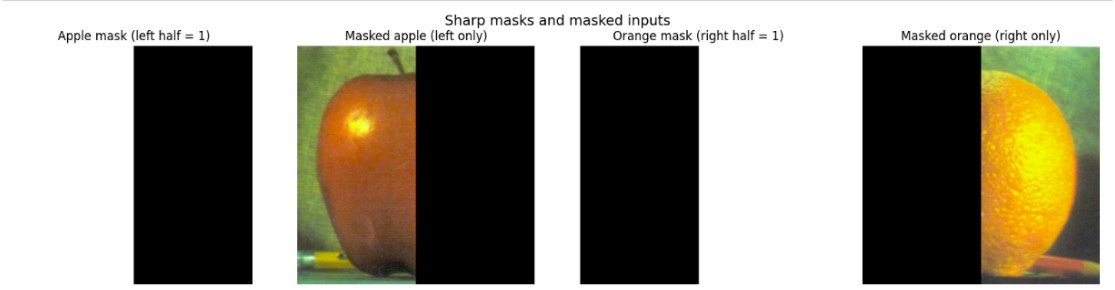
In this notebook we merge an apple (on the left) with an orange (on the right) so that the join looks natural — no visible straight line in the middle.

The idea is simple:

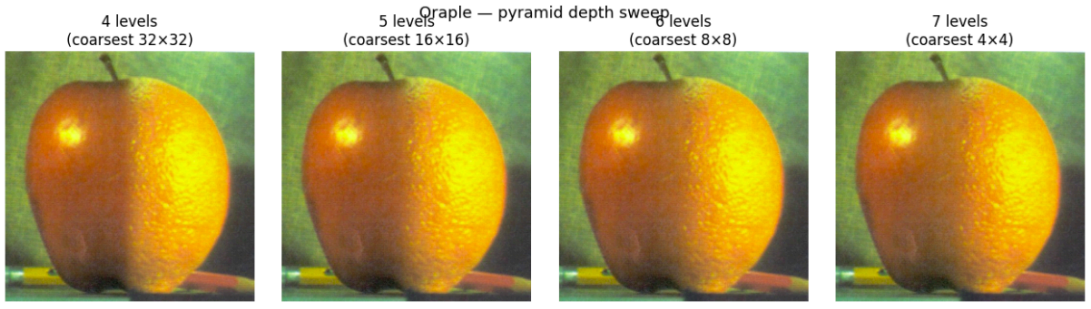
- A hard cut looks bad because it is a **step edge**. A step edge contains energy at *every* frequency, so the eye sees a sharp line no matter what you do with a single blur.
- The right fix is to split each image into **frequency bands** (coarse structure on one side, fine detail on the other) and blend each band over its **own width**: wide for coarse structure, narrow for fine detail.
- A **Laplacian pyramid** gives us those bands for free. It also gives us a **Gaussian pyramid of the mask** whose levels are already soft at the right width for each band.

The plan:

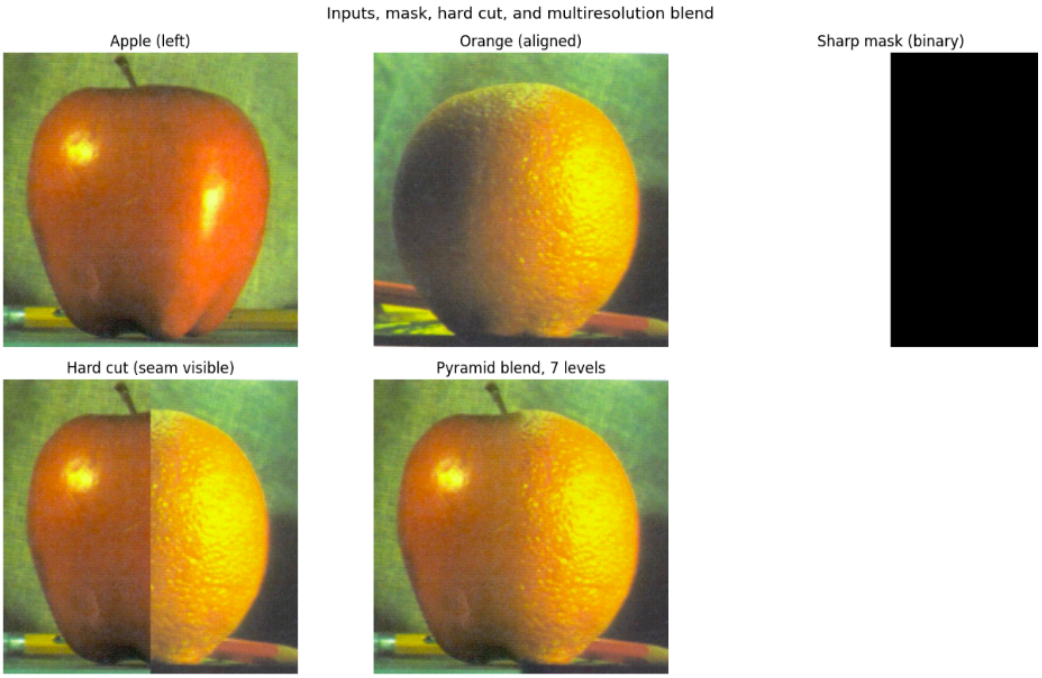
1. Load and align the two images.
2. Build a **sharp** binary mask (left half = 1, right half = 0).
3. Show what a hard cut looks like (as a baseline).
4. Build a Laplacian pyramid of each image and a Gaussian pyramid of the mask.
5. Combine level by level: `M·L1 + (1−M)·L2`.
6. Rebuild the image from the combined pyramid (collapse).
7. Show the seam for several pyramid depths and pick one.
8. For the chosen depth, show each level's contribution: `M·L1`, `(1−M)·L2` and their sum.



alt text



alt text



alt text

13. Why band-by-band hides the seam

A hard cut is a **step edge**. A step contains energy at every frequency, so you see a crisp line no matter how you blur a single scale.

The pyramid blend splits each image into bands and blends each band over its own width:

- **Fine bands** are blended over a couple of pixels. The eye is not sensitive to narrow changes at those scales, so no line is visible.
- **Coarse bands** (including the overall colour / brightness) are blended over tens or hundreds of pixels. A wide transition carries almost no low-frequency energy, so it looks like a gentle gradient.
- **Every band is matched to its own wavelength.** That is precisely what the mask's Gaussian pyramid gives us, level by level.

The result: no seam, no harsh colour bleed, and the two fruits still look like themselves.

Quick checks

Part 3: Seam Carving

Seam Carving: Content-Aware Image Resizing

This notebook shrinks an image's width by repeatedly removing the **lowest-energy vertical seam** (a connected path of pixels, one per row), as in Avidan & Shamir's seam carving paper.

Pipeline: 1. Set up paths and imports 2. Helper functions (channels, normalization) 3. Energy functions (gradient magnitude, DoG edges) 4. Seam finding with dynamic programming 5. Seam removal and the main `seam_carve` routine 6. Visualization and baseline resizing methods (subsample, crop) 7. Inspect the energy of a single image 8. Run everything on the full image set and save results

Energy functions

The energy map says how "important" each pixel is: seams are routed through low-energy (flat, boring) regions so that edges and objects survive.

Gradient-magnitude energy (`get_energy_grad_mag`): for each color channel, compute the gradient magnitude using the central-difference filter `Dx = [1, 0, -1]` (and its transpose for `Dy`), then **sum the three channels**.

DoG edge energy (`get_dog_energy`): for each channel, filter with Derivative-of-Gaussian filters, threshold the gradient magnitude automatically (`auto_threshold`), giving a binary edge map, then sum across channels. Higher `sigma` means smoother, coarser edges.

Finding the optimal vertical seam (dynamic programming)

`_find_vertical_seam` builds a cumulative cost matrix:

```
M[i, j] = e[i, j] + min(M[i-1, j-1], M[i-1, j], M[i-1, j+1])
```

Each row is computed in one vectorized step: the row above is shifted left and right (padded with `inf` at the borders) and the element-wise minimum of the three neighbors is added to the current row's energy.

The seam is then **backtracked**: start from the cheapest entry in the last row, and for each row going up, choose the cheapest of the three neighbors above. The result is an array of `H` column indices, one per row.

`_delete` removes one pixel per row using a boolean mask, so it works for 2D maps and `H x W x C` images alike.

Removing seams and the main `seam_carve` routine

`_remove_seams` repeats `k` times: find the best seam, record where it sat in the **original** image (using the `rows/cols` index maps that get cut along with the image), delete it from

the image/index maps/energy, and optionally **recompute the energy** with `energy_fn` after every removal (as the paper does).

`_gaussian_blur` is a small separable Gaussian blur (edge-padded). It isn't used by the pipeline below but is kept as an optional helper for smoothing energy maps.

`seam_carve` shrinks an image to `new_size`: 1. Build `rows/cols` index maps so every pixel remembers its original position. 2. Remove `W - new_w` vertical seams to reduce width. 3. Transpose the image and remove `H - new_h` vertical seams, which are horizontal seams in the original orientation, then transpose back.

It returns the list of removed seams (in removal order) and the resized image.

Visualization and baseline resizing methods

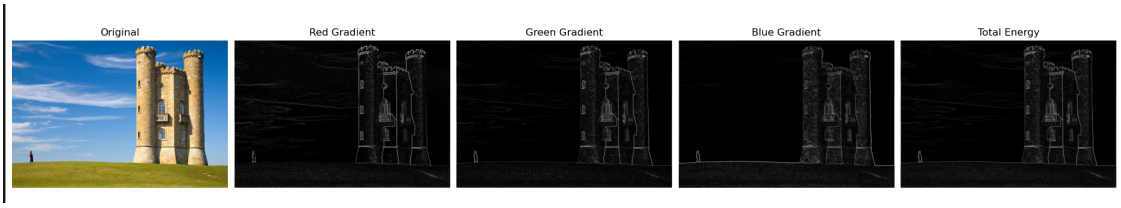
4. `draw_removed_seams`: paints every removed seam onto the **original** image (default red) in one vectorized step. This shows what the algorithm chose to delete.
5. `resize_subsample`: a naive baseline that resizes by **picking** one row/column from each block (no blending), so it squashes everything uniformly.

Comparing seam carving against subsampling and center-cropping shows why content-aware resizing is useful.

Inspecting the energy of one image

Before carving, it helps to see what the energy map looks like. This cell loads one image (`broadway_tower.png`), splits it into R/G/B channels, computes the per-channel gradient magnitudes with the `[1, 0, -1]` filter, and sums them into the final energy map.

Everything is normalized to `[0, 1]` for display. Bright areas in the energy map are edges and detail that seams will try to avoid.



alt text

Running seam carving on the full image set

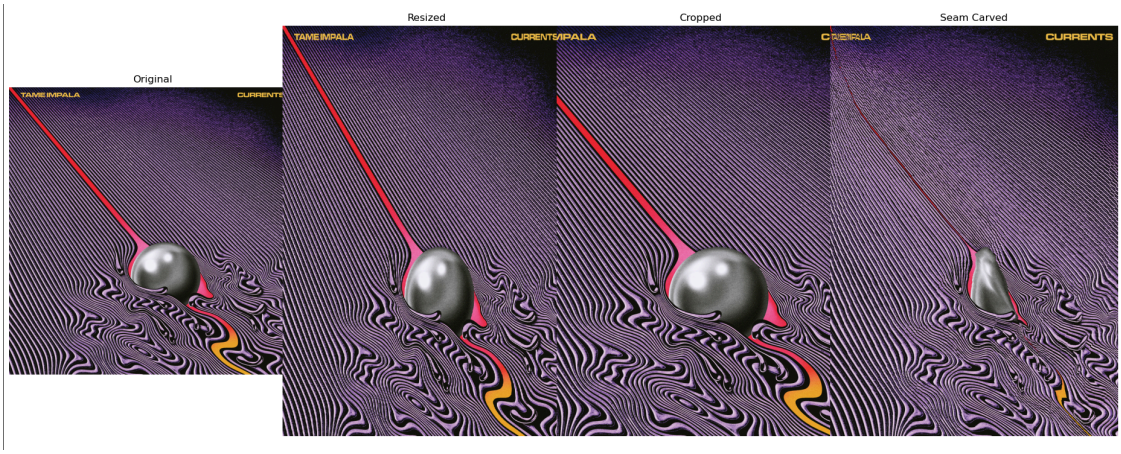
Configuration: - `SCALE`: fraction of the original width to keep (`0.7` removes 30% of the columns). - `ENERGY_TYPE`: `"finit_diff_energy"` (gradient magnitude) or `"dog_energy"` (DoG edges). The chosen function is used both for the initial energy and for recomputing it after every seam removal. - `SAVE` / `SHOW`: toggle writing results to disk and displaying them inline.

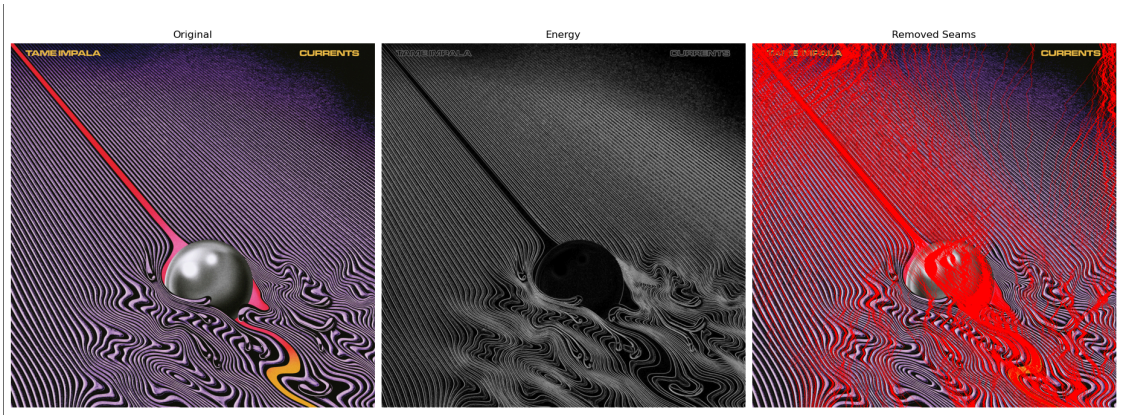
For each image the loop produces and compares four results: 1. **Seam carved**: the content-aware result 2. **Resized**: naive subsampling to the same size 3. **Cropped**: center crop to the same size 4. **Energy map** and **removed seams** (seams painted red on the original)

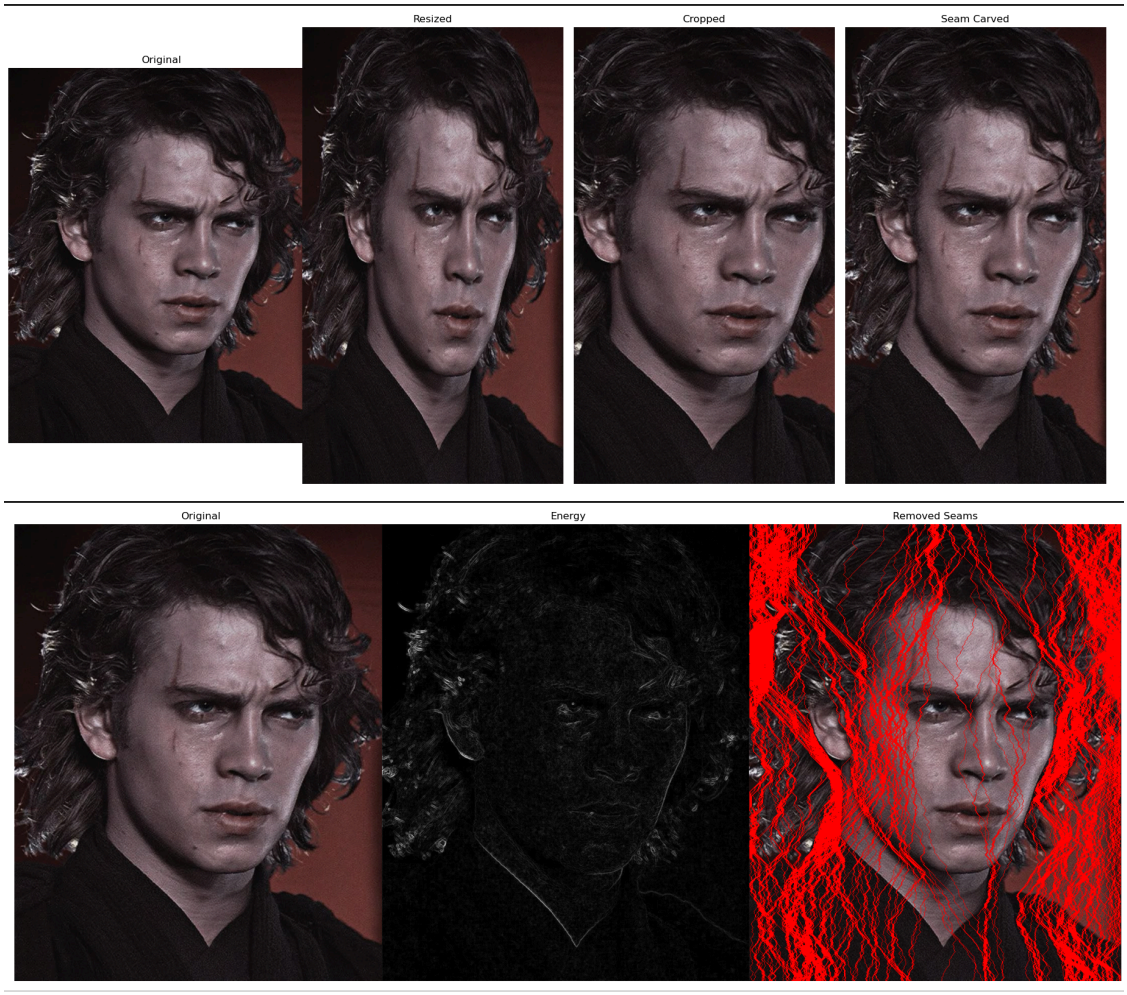
Outputs go to `images/<energy_type>/<image_name>/` next to the notebook. The energy map is normalized to `[0, 1]` before saving.

Seam removal is a Python loop over every seam, so this cell can take a while on large images.

alt text


Original
Resized
Cropped
Seam Carved


Original
Energy
Removed Seams



Why seam carving fails on faces and highly detailed images

Faces. Seam carving only sees *local gradient energy*, not meaning. A face has large smooth regions (cheeks, forehead, skin) that get low energy, so seams run straight through them. Removing those pixels squeezes the face while the high-energy edges (eyes, mouth, hair) stay in place. The result is distorted proportions, such as eyes too close together or a warped jawline. Humans are very sensitive to facial geometry, so even small distortions look wrong.

Images with lots of complex detail. When texture and edges are spread everywhere (foliage, crowds, starry skies, patterned fabric), the energy map is uniformly high and there is no cheap path for a seam to follow. The algorithm has to cut through important content, so it produces jagged, broken structures and visible artifacts. Straight lines and regular patterns (buildings, fences) get bent or interrupted because each seam shifts pixels by a different amount per row.

Root cause. Energy is a low-level proxy for importance. It assumes "smooth = unimportant", which is true for sky or grass but false for skin. Fixes include adding a

face-detection or saliency mask to the energy (high energy over protected regions), or using forward energy to reduce artifacts.

Bonus Part

Bonus: Object Removal and Seam Insertion

The Bonus makes an object disappear with seams, then widens the photo back to its original size by inserting seams.

Photo and mask

We used **broadway_tower.png** and removed **the man in the picture**. The mask has the same size as the photo, with white (1) over the object. We made it **dragged a box with --roi**. The program also grows the mask by 3 pixels, so the soft edge of the object cannot survive the removal.

Energy

The energy is the gradient magnitude of 1.2 on the grayscale image, with D_x = [1 0 -1] and D_y = D_x transposed, computed with our own 1.1 convolution (separable version) in float64. Following the instruction from Part 2 on, it uses reflect padding, so the border does not get an artificial frame response.

```
def energy(img):
    g = to_gray(img)
    dx = conv2d_separable(g, Dx, "reflect")
    dy = conv2d_separable(g, Dy, "reflect")
    return np.sqrt(dx ** 2 + dy ** 2)   # float64
```

Step 1: removing the object

On the masked pixels we set the energy to -10^6, so the cheapest seam is pulled through them. We then remove that seam from the photo and from the mask, recompute the energy on the new photo, and repeat until no masked pixel is left. The number of seams removed is k = **41**, so the photo went from width **714** to **673.**

```
def remove_object(img, mask):
    k = 0
    while mask.any():
        E = energy(img)
        E[mask] = -1e6
        seam = find_seam(E)
        img = remove_seam(img, seam)
        mask = remove_seam(mask, seam)
        k += 1
    return img, k
```

How the dynamic program finds the cheapest seam. A seam takes one pixel per row, and each pixel touches the one above it, directly or diagonally. We compute a table M, row by row. The first row is the energy itself. For every other pixel, M[i, j] = E[i, j] + min(M[i-1, j-1], M[i-1, j], M[i-1, j+1]), so M[i, j] is the minimum total energy of any seam that ends at pixel (i, j). The smallest value in the last row is the cost of the best seam. We trace it back from there: at each row going up, we step to whichever of the three

pixels above gave the minimum. Because of the large negative energy, this seam goes through the masked pixels.

Why trying every seam would take forever. A seam can step left, straight or right at every row, so there are about 3^(h-1) seams per starting column, an exponential number in the image height. The dynamic program reuses the best result for each pixel instead of recomputing it, so it costs only a few operations per pixel, about h x w in total.

Step 2: finding k seams on a copy

We make a copy of the reduced photo and remove k seams from it, recomputing the energy after each removal, as in 3.1. The copy is only used to find them. Next to the copy we keep an array that holds, for every pixel, its column in the reduced photo. We remove the same seams from this array, so after each seam we know where it sat in the reduced photo, and we mark those pixels in a boolean map.

```
def find_k_seams(img, k):
    h, w = img.shape[:2]
    copy = img.copy()
    cols = np.tile(np.arange(w), (h, 1))   # column in the reduced photo
```

```
    is_seam = np.zeros((h, w), dtype=bool)
    rows = np.arange(h)
    for _ in range(k):
        seam = find_seam(energy(copy))
        is_seam[rows, cols[rows, seam]] = True
        copy = remove_seam(copy, seam)
        cols = remove_seam(cols, seam)
    return is_seam
```

Step 3: widening the photo row by row

For each row we write every pixel once, except the pixels of the k seams, which we write twice. The first copy is averaged with its left neighbor and the second with its right neighbor. At the border we use the pixel itself as the missing neighbor. Every row has exactly k seam pixels, so every row grows by k and the photo returns to its original width. The inserted pixels are drawn in green.

```
left  = np.concatenate([img[:, :1], img[:, :-1]], axis=1)   # border: itself
right = np.concatenate([img[:, 1:], img[:, -1:]], axis=1)
first, second = (img + left) / 2.0, (img + right) / 2.0
# per row: seam pixels -> first copy then second copy; other pixels -> once
```

The program asserts that the final shape equals the original shape. In our run the output is **(484,714)**, equal to the original.

Why the k seams come from the copy

If we inserted the cheapest seam k times on the photo itself, the cheapest seam would be the same one every time. After inserting it, the new pixels are averages of their neighbors and have the same low energy, so the program would pick that same path again and again. The result would stretch one narrow strip of the photo into a wide smeared band. Removing the k seams from a copy avoids this: once a seam is removed, the next cheapest seam is a different path, so the k seams are spread over k different low-energy paths, and the photo is widened across many places by a small amount each. Keeping the column array lets us map each of those seams back to its position in the reduced photo, so all k can be inserted at once.